# Git & GitHub 실습

# Git & GitHub 실습 – 레포지토리 관리자

## 레포지토리 관리자(팀장)이 할 일

1. GitHub에서 레포지토리 생성(원격 레포지토리)

   오른쪽 상단 프로필 → Repositories → 초록색 New 버튼 → 이름 설정 후 하단의 Create Repository 버튼 누르기

   이때 GitHub 레포지토리의 기본 branch명은 main

2. 로컬에서 git 저장소 생성

   명령어: git init

   이때 로컬 레포지토리의 기본 branch명은 master로 생성됨(옛 git 기본값)

```
osy@OSY_PC MINGW64 /c/Work/rokey/practice_repository
$ git init
Initialized empty Git repository in C:/Work/rokey/practice_repository/.git/

osy@OSY_PC MINGW64 /c/Work/rokey/practice_repository (master)
$
```

# Git & GitHub 실습 – 레포지토리 관리자

3. 원격 저장소 연결(GitHub repository와 연결)

명령어: git remote add origin 깃허브주소

- origin: git clone 시 Git에서 자동으로 붙여주는 원격 저장소의 이름
- git remote add 원격저장소 별명 깃허브주소 ← 원격저장소 별명으로 origin이 아닌 다른 이름을 써도 된다.
- 원격 저장소 이름 변경도 가능하다.

이름 변경 명령어: git remote rename 원래이름 새로운이름

4. README 파일 생성

명령어: echo “# practice_repository” >> README.md

본 실습에서는 README 파일을 생성 및 작성하나 자유롭게 다른 파일을 작성해도 된다.

# Git & GitHub 실습 – 레포지토리 관리자

4. Staging(변경한 내용을 staging area에 추가)

특정 파일 추가 명령어: git add 파일명

모두 추가 명령어: git add .

5. Commit(staging area에 있는 변경사항을 local repository에 저장)

명령어: git commit -m “커밋메시지”

6. master 브랜치 이름을 main으로 변경

명령어: git branch -M main

branch: 브랜치 관리 git 명령어

- M: 강제로 이름 변경

main: 새 이름

★

master: Git에서 레포지토리 생성 시 기본 branch의 이름

main: GitHub에서 레포지토리 생성 시 기본 branch의 이름

master와 main의 기능 차이는 없으나 둘의 이름이 다르면
히스토리 충돌 문제가 발생하기 때문에 branch 이름을 맞춰준다.

# Git & GitHub 실습 – 레포지토리 관리자

4. GitHub 원격 레포지토리로 push

명령어: git push –u origin main

push: 원격 저장소에 업로드하는 git 명령어

-u: --set-upstream의 줄임말

upstream이란? 현재 브랜치가 기본적으로 데이터를 주고받을 원격 브랜치
즉, 이 브랜치가 앞으로 어디랑 push/pull할 지 연결해 둔 대상

명령어의 의미: 앞으로 현재 브랜치(로컬의 main)는 origin(원격저장소)의 main branch에 pull/push 하겠다

⬇

이후에 push/pull 할 때는 origin main을 명시하지 않고 git pull, git push만 쳐도
origin main으로 자동으로 pull/push된다.

# Git & GitHub 실습 – 개발자/팀원

## 개발자(팀원)이 할 일

1. 로컬 git 저장소 생성 + remote repository 연결 + remote repository 다운로드(동기화)

명령어1: git clone 레포지토리 주소

clone 후 상태:

```
osy@OSY_PC MINGW64 /c/Work/rokey/git_practice
$ git clone https://github.com/SeungyeonO/practice_repository.git
Cloning into 'practice_repository'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

osy@OSY_PC MINGW64 /c/Work/rokey/git_practice
$ ls
practice_repository/

osy@OSY_PC MINGW64 /c/Work/rokey/git_practice
$ cd practice_repository

osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (main)
$ ls
README.md

osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (main)
$
```

레포지토리명 확인 - 명령어2: ls

명령어3: cd 레포지토리명

git clone을 한 경로에 원격 저장소와 같은 이름으로 git 저장소가 생성됨

로컬 저장소에 원격 저장소 내용이 그대로 반영됨
기본 branch도 master가 아닌 main!

# Git & GitHub 실습 – 개발자/팀원

2. branch 생성

명령어: git checkout -b 브랜치명

[브랜치명] branch를 생성하고 그 branch로 이동

```
osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (main)
$ git checkout -b cloneBranch
Switched to a new branch 'cloneBranch'

osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (cloneBranch)
$
```

3. 파일 작성 후 로컬 레포지토리에 반영

명령어1: git add .

명령어2: git commit -m “커밋메시지”

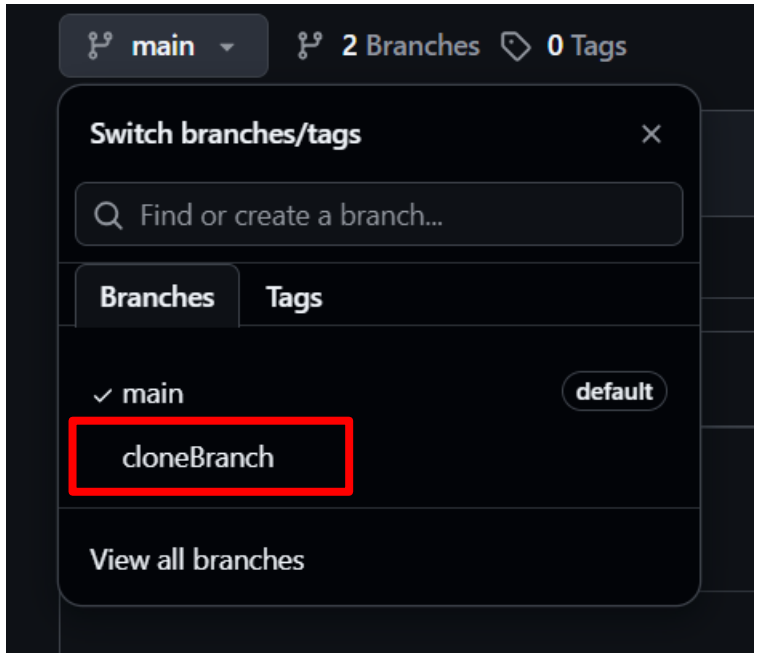
```
osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (cloneBranch)
$ vi document.txt

osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (cloneBranch)
$ ls
README.md  document.txt

osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (cloneBranch)
$ git add .
warning: in the working copy of 'document.txt', LF will be replaced by CRLF the next
time Git touches it

osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (cloneBranch)
$ git commit -m "add document"
[cloneBranch f6a4fdd] add document
 1 file changed, 1 insertion(+)
 create mode 100644 document.txt
```

# Git & GitHub 실습 – 개발자/팀원

4. branch 원격 레포지토리에 push

명령어: git push origin 브랜치명

```
osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (cloneBranch)
$ git push origin cloneBranch
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 298 bytes | 298.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'cloneBranch' on GitHub by visiting:
remote:      https://github.com/SeungyeonO/practice_repository/pull/new/cloneBranch
remote:
To https://github.com/SeungyeonO/practice_repository.git
 * [new branch]      cloneBranch -> cloneBranch
```




이때 로그인 창이 뜨면
username: 본인의 GitHub 아이디
password: 팀장이 제공한 personal access token
위와 같이 입력한다.

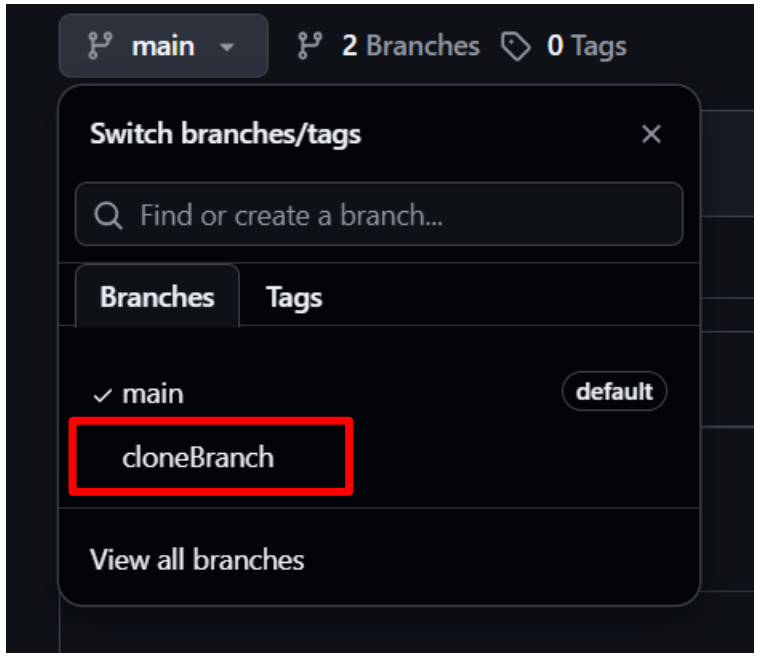
GitHub 원격 레포지토리에서
branch가 생긴 것을 확인 가능

# Git & GitHub 실습 – 개발자/팀원

4. branch 원격 레포지토리에 push

명령어: git push origin 브랜치명

```
osy@OSY_PC MINGW64 /c/Work/rokey/git_practice/practice_repository (cloneBranch)
$ git push origin cloneBranch
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 298 bytes | 298.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'cloneBranch' on GitHub by visiting:
remote:      https://github.com/SeungyeonO/practice_repository/pull/new/cloneBranch
remote:
To https://github.com/SeungyeonO/practice_repository.git
 * [new branch]      cloneBranch -> cloneBranch
```




이때 로그인 창이 뜨면
username: 본인의 GitHub 아이디
password: 팀장이 제공한 personal access token
위와 같이 입력한다.

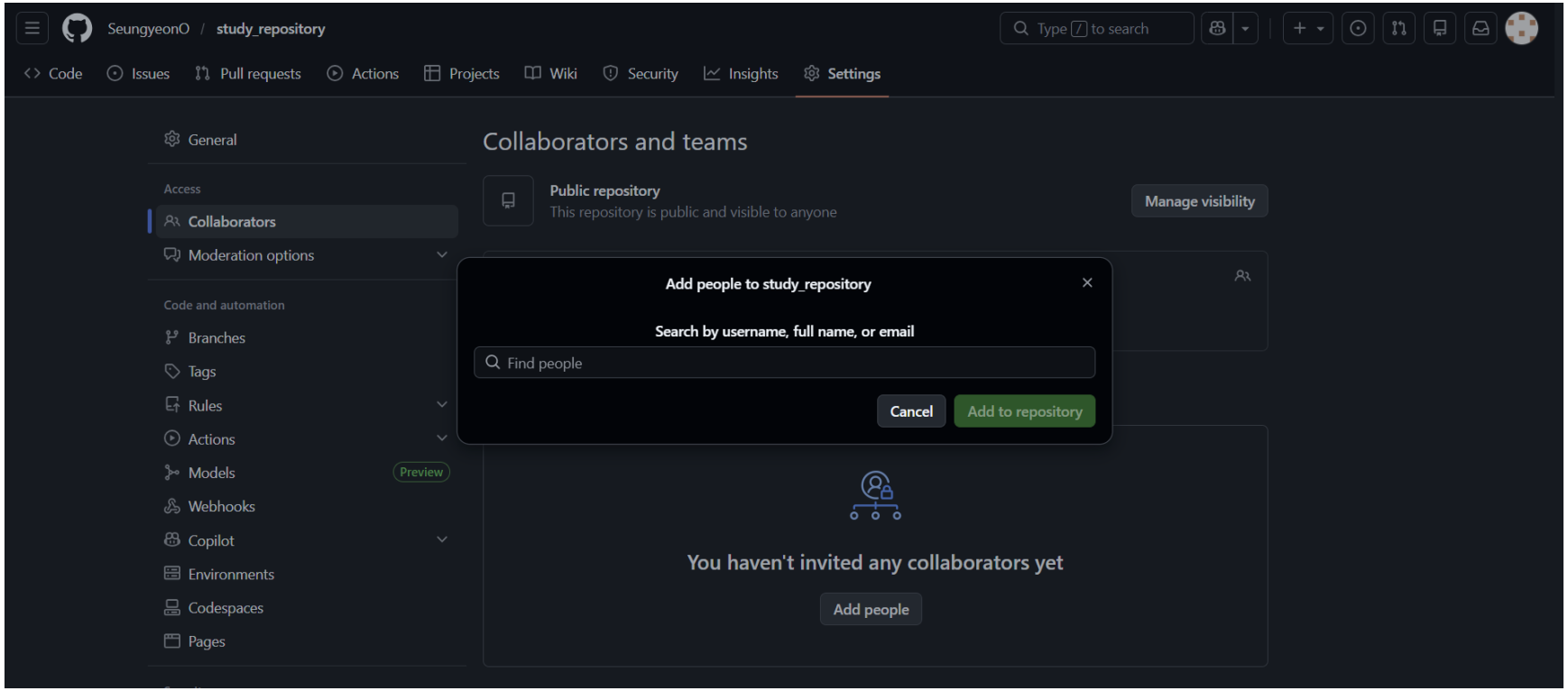
GitHub 원격 레포지토리에서
branch가 생긴 것을 확인 가능

# Git & GitHub 실습 – pull request

## 5. pull request

pull request 권한 획득 방법1(team repository방식): 팀장이 팀원들을 collaborator로 추가한다.

팀장: 해당 repositor의 Settings → Collaborators → Manage Access의 Add people 버튼 → 팀원 아이디 입력 후 추가



# Git & GitHub 실습 – pull request

5. pull request

pull request 권한 획득 방법2(큰 조직에서 사용): Organization 기반 협업

대표자: Organization 생성 → 팀 생성 → repository 생성 → 팀에 repository 권한 부여

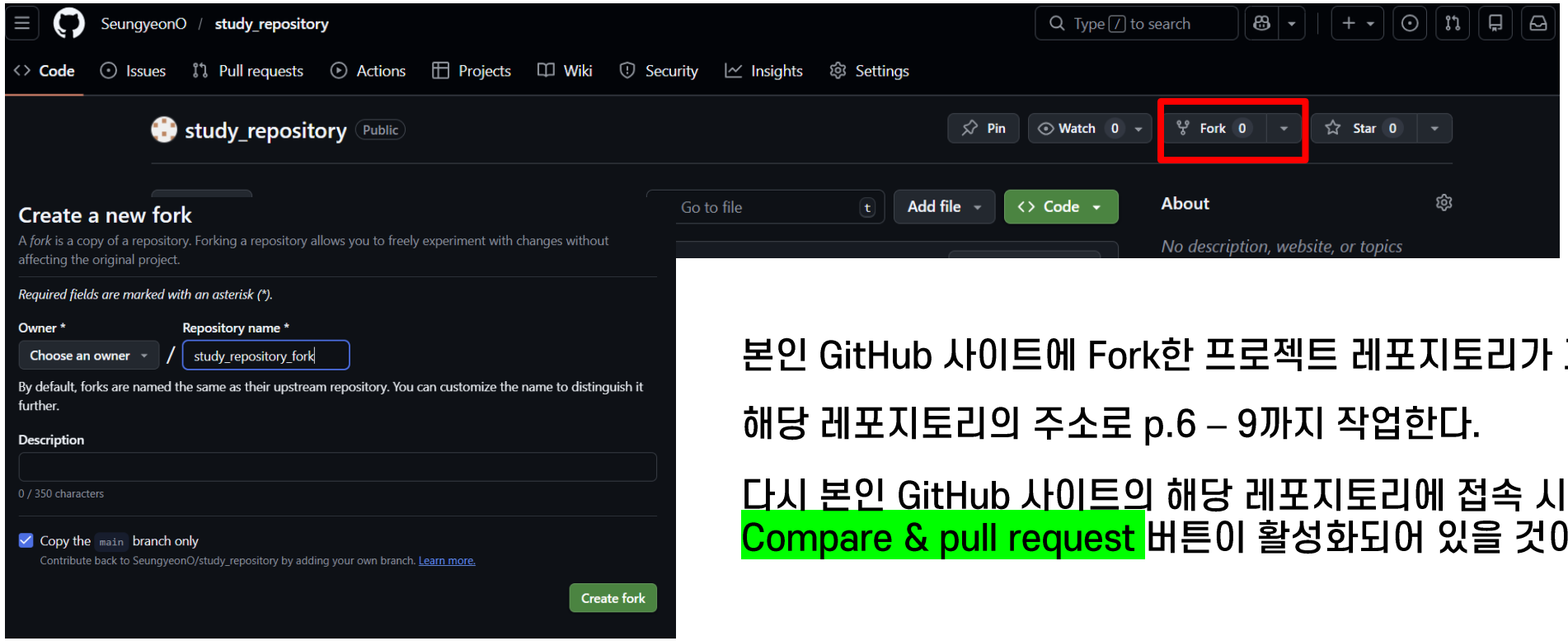
추후 보강

https://codingtoday.tistory.com/86

# Git & GitHub 실습 – pull request

## 5. pull request

pull request 권한 획득 방법3(오픈소스 프로젝트 방식): Fork 기반 협업




본인 GitHub 사이트에 Fork한 프로젝트 레포지토리가 그대로 복사되어 있다.

해당 레포지토리의 주소로 p.6 – 9까지 작업한다.

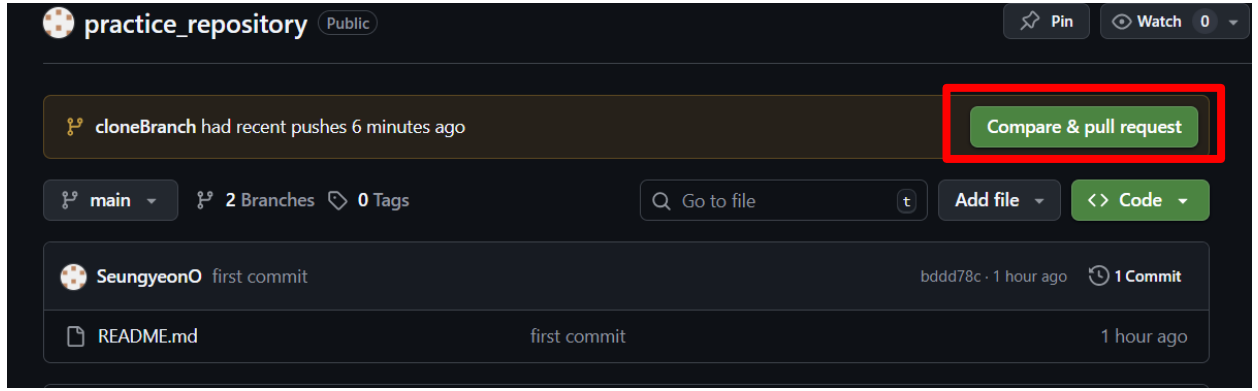
다시 본인 GitHub 사이트의 해당 레포지토리에 접속 시
Compare & pull request 버튼이 활성화되어 있을 것이다.

https://velog.io/@sagesrkim/Github-fork%EB%A1%9C-%ED%8C%80-%ED%94%84%EB%A1%9C%EC%A0%9D%ED%8A%B8-%ED%98%91%EC%97%85%ED%95%98%EB%8A%94-%EB%B0%A9%EB%B2%95

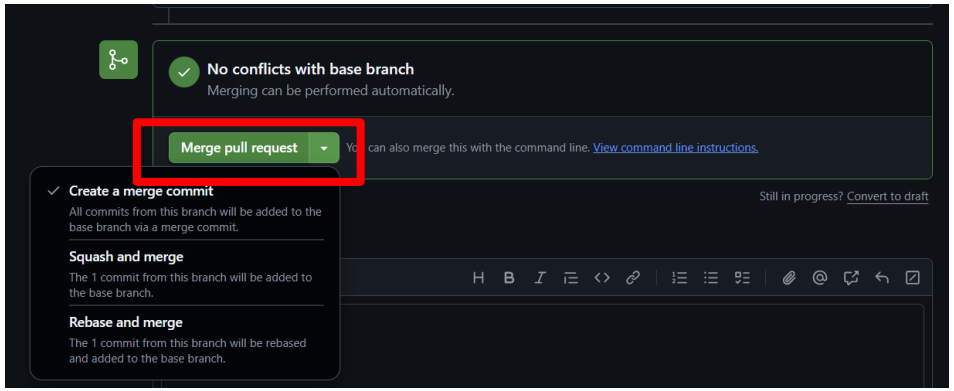
# Git & GitHub 실습 – pull request

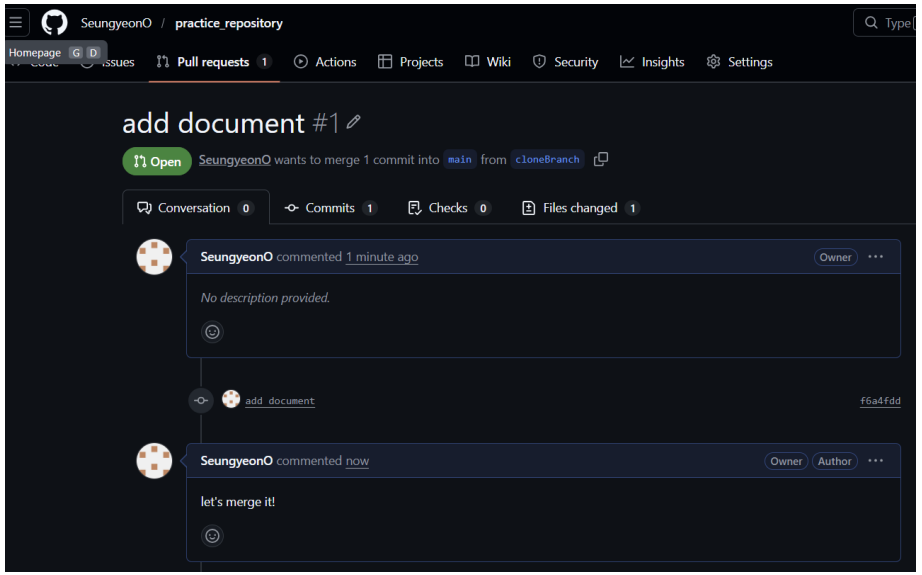
## 5. pull request

팀원: Compare & pull request 버튼 선택해 메시지를 작성하고 PR 보내기




관리자: PR을 받으면 변경 내역 확인, 코드 리뷰 후 merge 여부를 결정



# Git & GitHub 실습 – trouble shooting

## 1. Push & pull request 권한 문제

push 시 token을 요구하고 팀원들에게 pull request 버튼이 활성화 되지 않는 문제

해결: 팀장이 팀원들을 repository의 collaborator로 추가하여 권한을 부여한다.

* Collaborator로 추가하지 않아도 토큰이 있으면 push가 가능하지만 토큰 공유는 보안상 매우 위험한 방법이라고 함

## 2. 히스토리(commit 내역) 충돌 문제

git clone 이용시:

- 로컬 저장소 생성
- 원격 저장소와 연결
- 원격 저장소의 히스토리를 다운받아 동기화

자동으로 한꺼번에 실행

git init 이용시:

```
git init
git remote add origin 깃허브주소
```

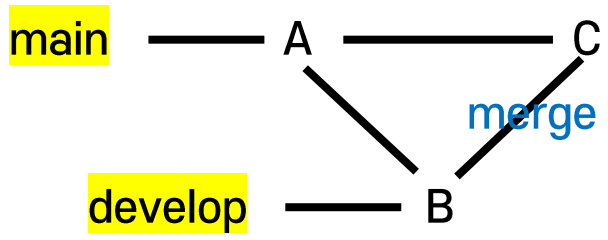
← 로컬 저장소 생성
원격 저장소와 연결

```
git pull origin main
git branch –M main
```

← 다운로드와 동기화

# Git & GitHub 실습 – trouble shooting

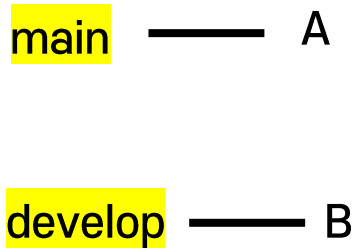
git clone 이용 / git init 방법에서 다운로드 및 동기화까지 마친 후 branch 생성:




A: README commit

B: document commit

git init 방법에서 다운로드 및 동기화 안 하고 branch 생성:




두 브랜치는 아예 다른 프로젝트라고 봐도 무방 → merge 할 수 없음

# Git & GitHub 실습 – 각자 스터디 자료 관리용

협업 목적으로 branch를 생성해 개발 후 main에 merge하는 용도가 아닌,

스터디에서 각자 공부한 내용을 업로드하는 작업 공간으로 쓸 것이라면

초기 설정(첫 자료 업로드 시)

```
git init
git remote add origin 깃허브주소
git checkout -b 브랜치명
git add .
git commit -m “커밋메시지”
git push origin 브랜치명
```

다음 번 학습 자료 업로드 시

```
git add .
git commit -m “커밋메시지”
git push origin 브랜치명
```

브랜치명은 본인 이름으로 설정해주세요!!